System Architecture of Portable Guitar MultiFX

Javier Estiven León Calderón
Electronic Engineering
Universidad Nacional de Colombia

***Abstract*—Portable Guitar MultiFX is a proposed compact practice amplifier combining guitar input conditioning, digital effects, simple controls, and loudspeaker reproduction. Following the architectural views in Chapter 2 of Elecia White's *Making Embedded Systems*, this report defines the product's external interactions, decomposes its implementation into functional blocks, and organizes the proposed controller software by responsibility and dependency. The architecture separates continuous audio processing from user-interface control and distinguishes the product enclosure from the main PCB. Component restrictions are established project constraints; interfaces, autonomous startup, and complete-system performance still require verification. These diagrams propose a design, not a validated prototype.**

***Index Terms*—Embedded systems, system architecture, guitar effects, DSP, hardware–software integration.**

I. Purpose and Architectural Scope

White's context, block, and organigram views answer different questions: how the product interacts with its environment, what implements its functions, and which software responsibilities depend on others [1]. The following views retain the same system boundary and functional intent.

The project aims to provide guitar practice with a compact, self-contained amplifier. Its initial functional target is mono audio with clean and drive modes, equalization, two buttons, LED feedback, and parameter adjustment through the DSP module's potentiometers. After programming, normal operation should require an external power supply but no computer.

The proposed audio platform is a TSA1702B module based on the ADAU1701. The module documentation describes analog audio interfaces and SigmaStudio programming [2]; the processor documentation identifies integrated audio converters, a programmable DSP, and control interfaces [3]. A separate STM32F030C8T6 is proposed for user-interface and DSP-parameter control. The STM32 family provides GPIO, timing, I^2C, and SWD facilities [4], but their presence does not establish a working connection to the actual DSP board.

The main PCB is restricted to components from the professor's Altium library. Other physically obtainable components must connect externally through headers; use of the DSP module is accepted. Here, *external to the PCB* can still mean *inside the product*. Amplifier, speaker, conditioner, connector, and supply selections remain open within those constraints.

II. Context: Product and Environment

Fig. 1 treats the complete amplifier as a black box. The guitar supplies an electrical instrument signal. The guitarist selects and adjusts the sound and receives both acoustic output and visual feedback. The controls and loudspeaker belong to the product, so they are not represented as external devices.

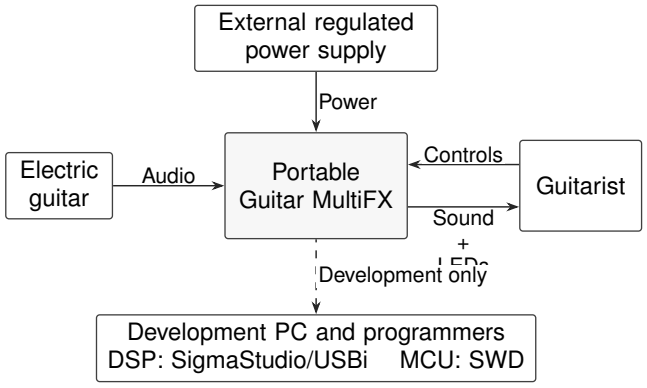


Fig. 1. Proposed product context. The central box includes the speaker, controls, main PCB, and internal modules. The dashed development link represents configuration and debugging, not a runtime dependency.

The development equipment configures the DSP and programs or debugs the controller. USBi and SWD serve different devices and are not interchangeable programmers. Their depiction describes the intended development workflow, not confirmed availability of every tool or a verified programming pinout. The final product must retain the intended DSP program and start without that equipment.

The supply is intentionally specified by function rather than a fixed voltage or current. Its final ratings depend on the chosen amplifier, regulation, actual module inputs, and measured loads. Battery operation, a display, and additional effects are outside the initial target.

III. Blocks: Audio, Control, and Power

Fig. 2 decomposes the product while keeping audio, control, and power distinct. The conditioner provides an appropriate load for guitar pickups and adapts the signal to the module input. Its gain, bias, protection, and usable input range must be selected together; a connector alone does not provide this adaptation.

The ADC, DSP, and DAC functions belong to the ADAU1701 [3]. The proposed SigmaStudio program supplies clean routing, drive, EQ, level management, and smooth parameter changes. These functions and the potentiometer assignments must be implemented and checked in the actual program. The module's analog output is a signal-level interface to a separate amplifier; the power stage, rather than the DSP output, drives the loudspeaker.

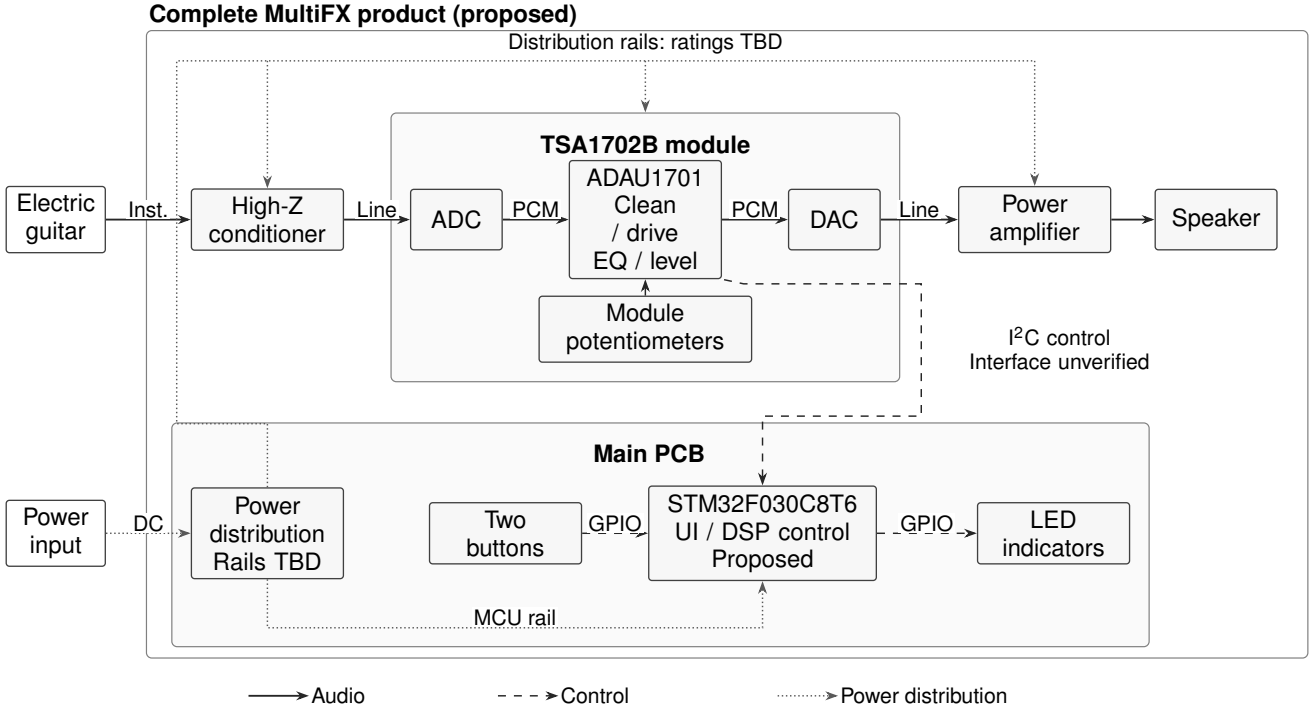


Fig. 2. Proposed functional implementation. ADC, DSP, and DAC are logical functions of the ADAU1701, not three additional ICs. The speaker and external modules remain inside the product; links crossing the main-PCB boundary require defined headers. Dotted routes denote supply distribution and may represent different regulated rails, not one electrically common net. Audio after the amplifier drives the speaker.

The STM32 handles buttons, indicators, and control transactions. It does not transport audio samples. The module potentiometers act on DSP parameters directly through the module's control facilities. Their mapping is independent of the controller's button sampling.

An I^2C link is proposed, but its access on the actual TSA1702B, logic levels, addressing, and interaction with USBi remain unverified. The ADAU1701 supports EEPROM self-boot [3]; controller transactions must respect that sequence. Runtime control must use the parameter addresses and formats exported for the matching SigmaStudio program.

The dotted supply routes express responsibility for powering each active subsystem. They are not a wiring schematic: a common distribution source does not imply identical voltages at all loads. Connector ratings, regulation, returns, decoupling, and transient demand require a separate electrical design.

IV. Organigram: Software Responsibilities

Fig. 3 applies White's organigram as a hierarchy of software responsibilities and dependencies [1]. It is a proposed organization of the controller software, not an execution sequence or an organization of people.

The application coordinator establishes startup order, maintains the operating mode, requests parameter changes, and determines the visible response to communication errors. Button handling produces debounced events; indicator handling converts application state into LED patterns. Both depend on board-specific GPIO and timing support. Sharing that support does not authorize competing ownership of the same output pin.

The DSP control service separates the requested musical behavior from device-specific parameter representations. A matching export defines addresses and numerical encoding; the transport supplies bounded transactions and explicit errors. GPIO, timing, and transport initialization belong to board support, with startup coordinated by the application. This view identifies responsibilities without fixing a framework, RTOS, file-per-box implementation, or final scheduling method.

The hierarchy also defines an error contract: a transport error returns to the application through the DSP service. The controller can indicate a persistent fault and suspend parameter changes. It cannot guarantee a mute or a return to clean through a failed control link. Such behavior would require a separately defined and verified mechanism.

V. Consistency and Validation

The three views describe one product: external interactions in Fig. 1, functional implementation in Fig. 2, and controller-software dependencies in Fig. 3. The DSP audio program and MCU firmware are separate deployments linked by a versioned parameter contract. The main PCB integrates permitted components and module interfaces; it does not duplicate the entire commercial DSP board.

Table I defines verification work rather than reporting results. The module and USBi are reported available; no complete audio-chain test, deployed controller firmware, fabricated MultiFX PCB, or validated system power budget is recorded.

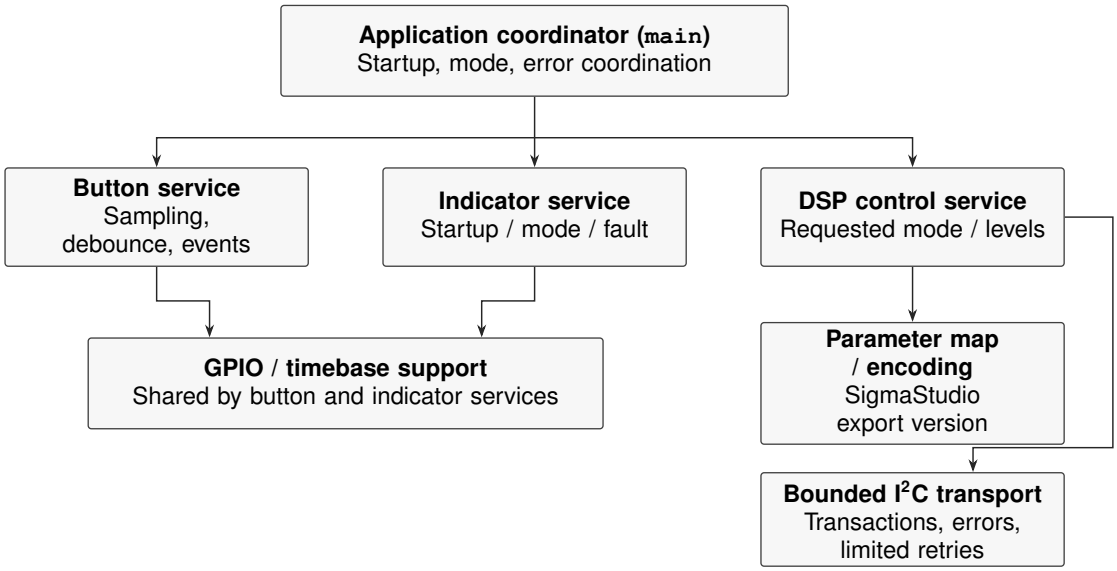


Fig. 3. Proposed controller-software organigram. Downward arrows denote dependencies, not audio flow or temporal order. Lower services return events, status, and errors to their callers. The shared GPIO/timebase block exposes a common platform dependency; the separately deployed SigmaStudio audio program is outside this hierarchy.

TABLE I
INTERFACE REQUIREMENTS STILL TO VERIFY

Interface	Required evidence before design release
Guitar to ADC	Actual input impedance, bias, gain, accepted signal range, and unclipped operation with the conditioner.
DSP to amplifier	Compatible output level, input sensitivity, gain, speaker load, and measured clean output.
External DSP control	Accessible bus, logic levels, addressing, parameter format/version, and successful commands on the actual board.
Power and startup	Correct rails, load/transient budget, repeatable autonomous boot, and coordination of DSP, MCU, and programmer access.

Manufacturer capabilities are design inputs, not measured prototype specifications.

Before committing to the PCB, identify the actual board and native library, demonstrate clean audio, verify autonomous startup without a computer, and measure the effect of an externally commanded DSP parameter change. Record hardware identity, software/export versions, connections, signal conditions, expected behavior, and observed results. These tests reduce the critical integration risks while retaining the component boundary and keeping unproven performance claims out of the design.

REFERENCES

[1] E. White, *Making Embedded Systems: Design Patterns for Great Software*, 2nd ed. O'Reilly Media, 2024, ch. 2.

[2] TinySine, "TSA1702B: Bluetooth 5.1 + DSP Audio Receiver Board," datasheet. Accessed: Oct. 20, 2026. [Online]. Available: https://www.tinyosshop.com/datasheet/TSA1702B_Datasheet.pdf

[3] Analog Devices, "ADAU1701: SigmaDSP 28-/56-bit audio processor with two ADCs and four DACs," datasheet, rev. C, 2016. [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/ADAU1701.pdf

[4] STMicroelectronics, "STM32F030x4/x6/x8/xC," datasheet DS9773, rev. 5, Nov. 2021. [Online]. Available: https://www.st.com/resource/en/datasheet/stm32f030c8.pdf